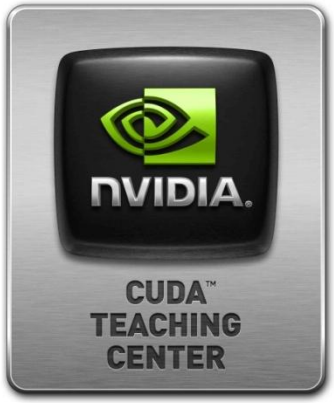
Programación masivamente paralela en procesadores gráficos (GPUs)

E. Dufrechou, M. Freire, P. Ezzatti y M. Pedemonte



Clase 5
Programación CUDA

Contenido

- **Modelo de programación CUDA**
  - **Introducción**
- **Programación en CUDA**
  - **Conceptos básicos**

Modelo de programación CUDA

Introducción

La GPU:

- Es un coprocesador (device) de la CPU (host).
- Posee su propia memoria DRAM (memoria del device), al menos en términos lógicos (GPUs integradas, SoCs).
- Ejecuta “muchos” threads en paralelo.
- Existen otros coprocesadores (TPUs, etc)

Una porción de los cálculos con paralelismo de datos se computan en paralelo con “muchos” hilos en el device en rutinas llamadas kernels.

Introducción - Hilos

Diferencias entre los threads de la GPU y CPU

1. **Costo de los hilos:**
   a. **CPU: Costosos, manejan lógica compleja**
   b. **GPU: Livianos, poco overhead para crearlos**

2. **Cantidad**
   a. **CPU: Generalmente se usan pocos hilos (orden de #cores)**
   b. **GPU: Se requieren miles de hilos para hacer cómputo eficiente**

Introducción - Programas CUDA

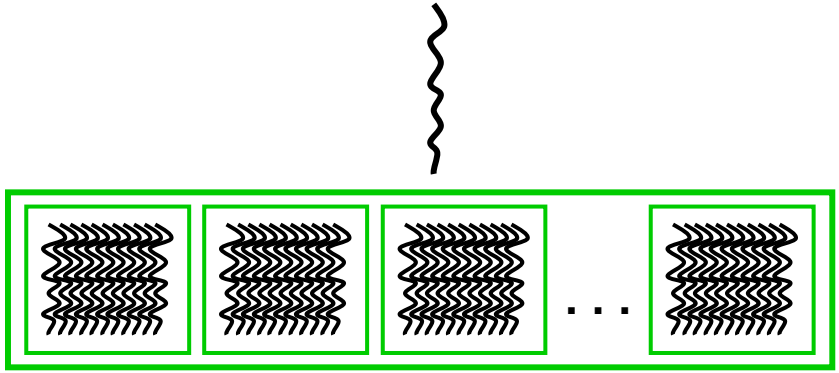
Los programas en GPU integran código en el host con código en la GPU

- **El código del host puede ser secuencial o paralelo.**
- **En la tarjeta el código es altamente paralelo (SPMT).**

Código en el host (secuencial)

Kernel device (paralelo)

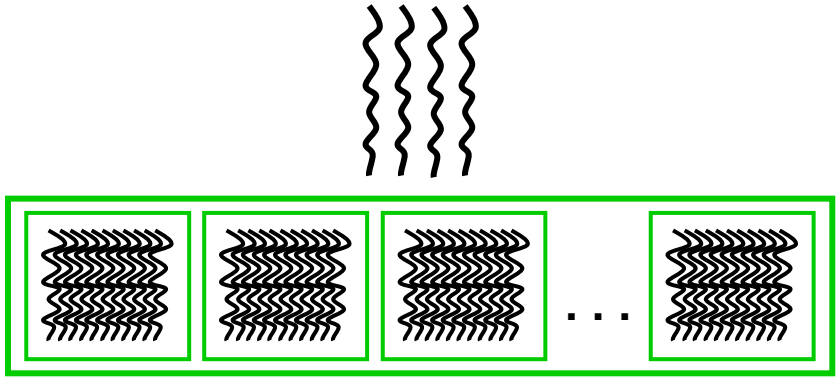
KernelA<<< nBlk, nTid >>>(args);



Código en el host (paralelo)

Kernel device (paralelo)

KernelB<<< nBlk, nTid >>>(args);



Introducción - Programas CUDA

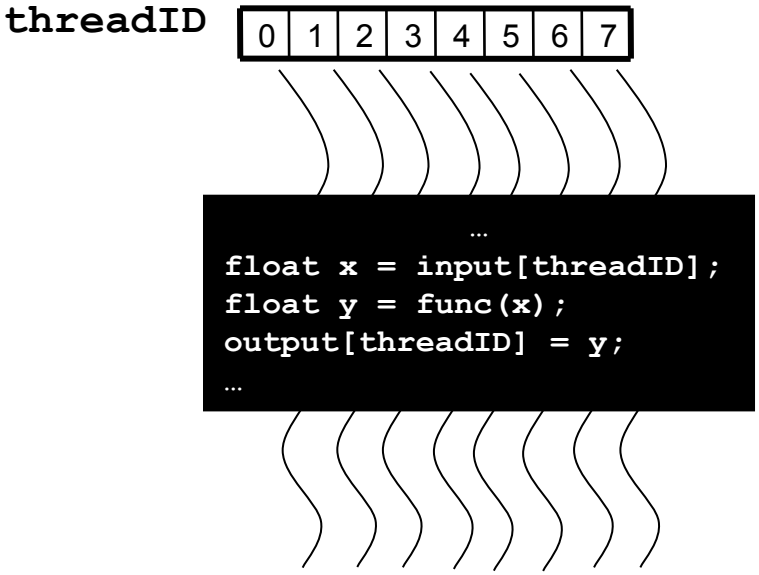
Algoritmo básico

1. Instrucciones en el host
2. Se envían los datos del host a la GPU
3. Se procesa en GPU
4. Se recuperan los resultados al host
5. Continúan instrucciones en el host

Introducción - Organización de threads

¿Cómo se ejecuta un kernel?

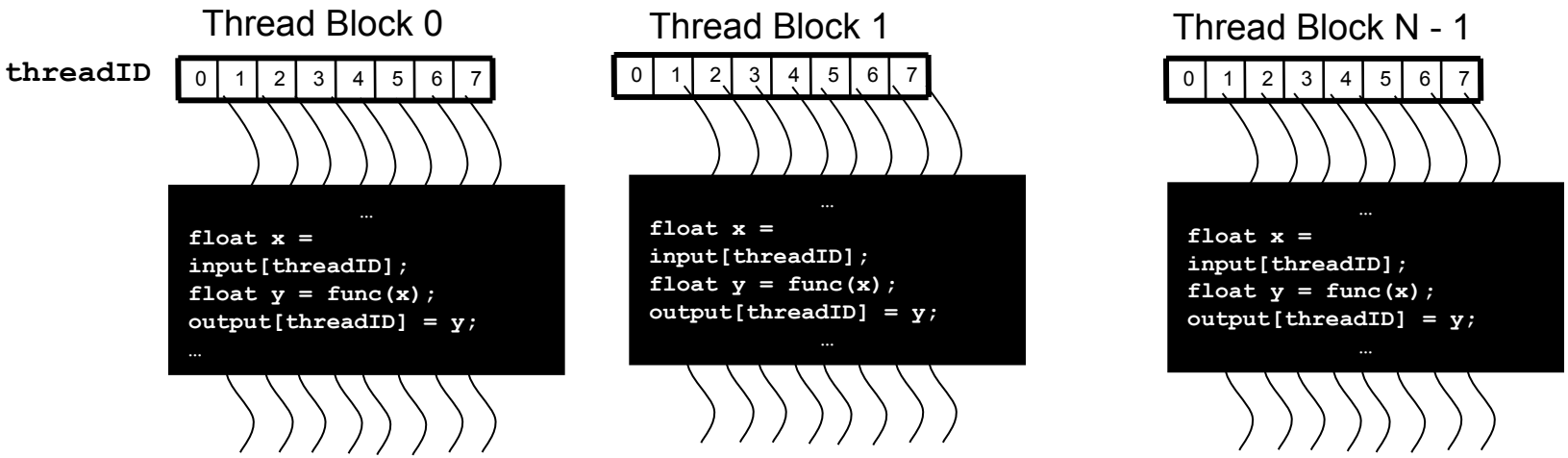
- Es ejecutado por un array de threads
  - Todos los threads corren el mismo código (SPMT)
  - Cada thread tiene un ID que se puede utilizar para computar direcciones de memoria y tomar decisiones de control



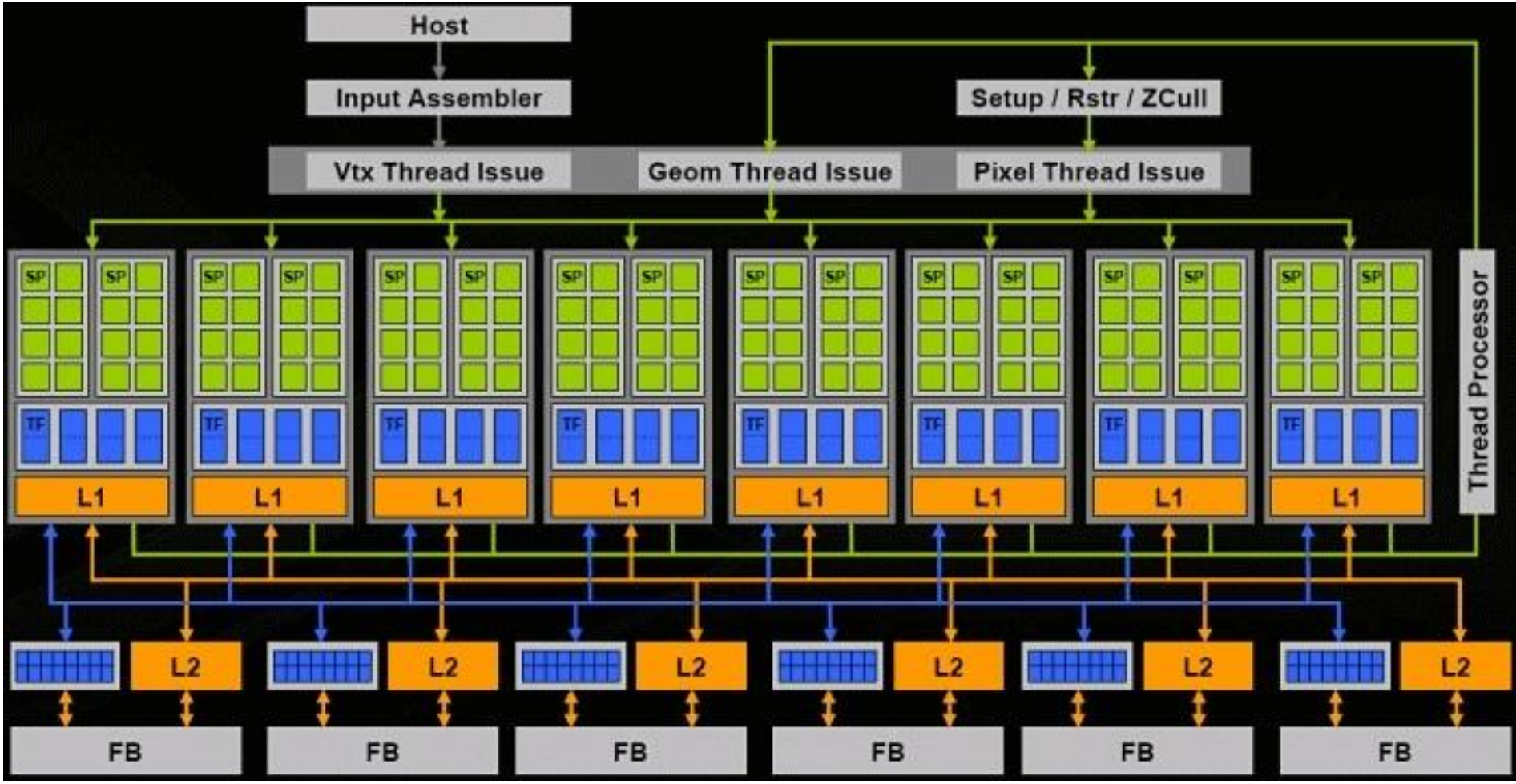
Introducción - Organización de threads

Los hilos se agrupan en bloques

- Facilita la escalabilidad
- Permite dividir un array de threads en múltiples bloques:
  - Threads en un mismo bloque cooperan via **memoria shared, operaciones atómicas** y **barreras de sincronización**.
  - Threads en diferentes bloques no pueden cooperar directamente (requiere usar mem global - operaciones atómicas).



Introducción - Organización de threads



Introducción - Organización de threads

¿Cómo se organiza CUDA?

- Los hilos se agrupan en bloques 3D (2D hasta Fermi)
- Cada bloque ejecuta de manera independiente, en algún multiprocesador de los demás (y en un orden indeterminado)
- Cada kernel genera una grilla tridimensional de bloques
  - Originalmente una única grilla a la vez
  - En las tarjetas más recientes es posible tener más de un kernel a la vez en paralelo de manera independiente (no se ve en el curso)

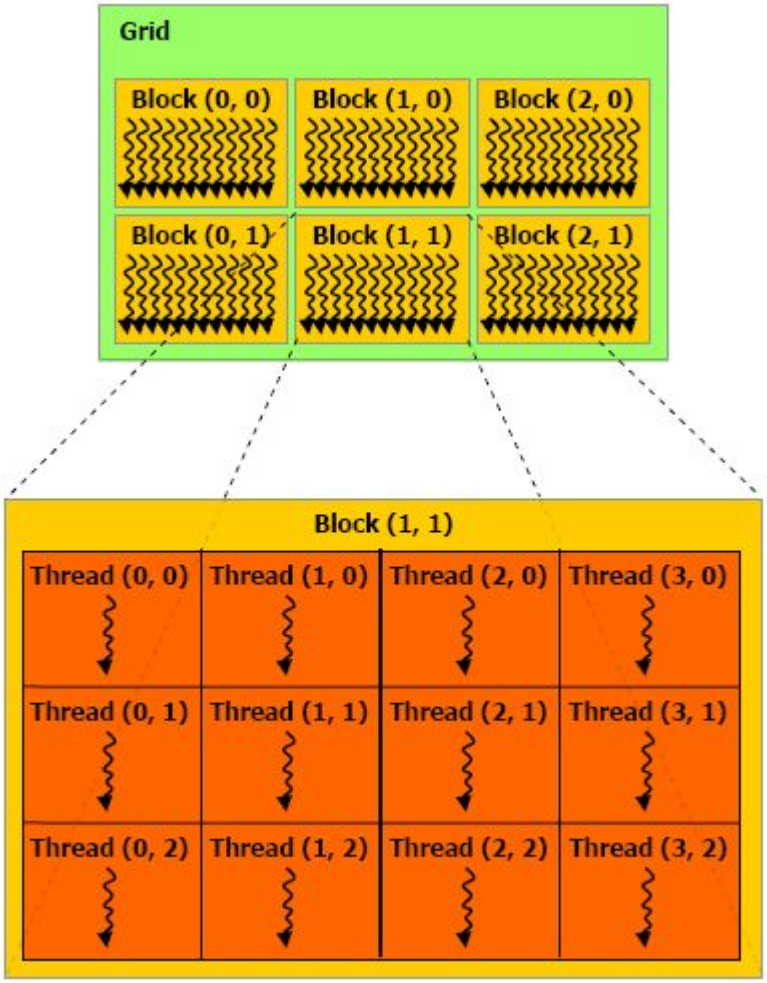
Introducción - Organización de threads

Jerarquía de threads

- **Los bloques dentro de un grid tienen que ser independientes.**
- **Los bloques de threads son ejecutados en cualquier orden y en diferentes multiprocesadores.**
- **En las tarjetas nuevas hay ejecución concurrente de kernels (en el curso no lo vamos a tener en cuenta !!!).**

Introducción - Organización de threads

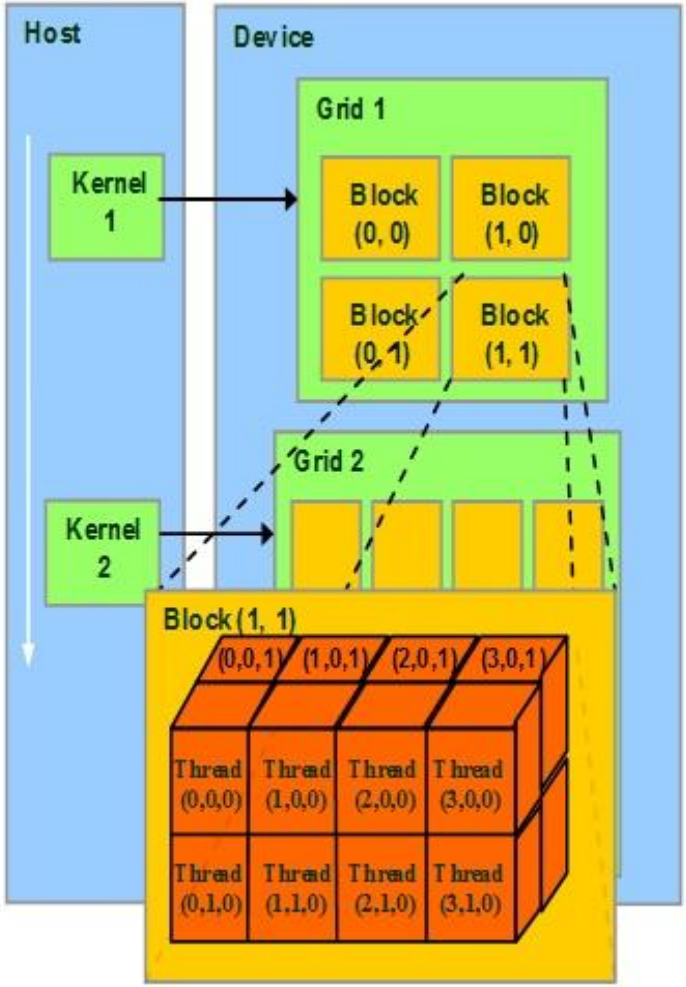
Grilla bidimensional con bloques bidimensionales



Introducción - Organización de threads

Direccionamiento de threads

- Se usan dos trios de índices para identificar threads (y decidir en qué datos trabajan)
  - Block ID: 1D, 2D o 3D (3D desde Fermi)
  - Thread ID: 1D, 2D o 3D
- Este formato facilita el direccionamiento de memoria en muchas aplicaciones
  - Procesamiento de imágenes.
  - Resolución de PDEs (regulares) en 3D.



Programación en CUDA

Programación en CUDA - Tipo de funciones

Las funciones

	Ejecutable en:	Invocable desde:
`__device__ float DeviceFunc()`	device	device
`__global__ void KernelFunc()`	device	host
`__host__ float HostFunc()`	host	host

- `__global__` define funciones "kernel"
- `__host__` puede no ponerse.
- `__device__` y `__host__` pueden ser utilizados juntas (con compilación condicional).

Programación en CUDA - Ejecución de kernel

¿Cómo se ejecuta una rutina en cuda?

```
__global__ void KernelFunc(int a, int b){...}

main(){

            /*  Código en C++  */

   dim3   DimGrid(100, 50);     // 5000 bloques
   dim3   DimBlock(4, 8, 8);    // 256 threads por block
   size_t SharedMemBytes = 64; // 64 bytes de memoria shared
   KernelFunc<<< DimGrid, DimBlock, SharedMemBytes >>>(10,37);
   cudaDeviceSynchronize();

            /*  Código en C++  */
}
```

Programación en CUDA - Ejecución de kernel

¿Cómo se ejecuta una rutina en cuda?

defino el kernel

```
__global__ void KernelFunc(params){...}

main(){
            /*  Código en C++  */
   //reservar y transferir memoria a p

   dim3   DimGrid(100, 50  );    // 5000 bloques
   dim3   DimBlock(4, 8, 8 );   // 256 threads por block
   size_t SharedMemBytes = 64; // 64 bytes de memoria shared
   KernelFunc<<< DimGrid, DimBlock, SharedMemBytes >>>(params);
   cudaDeviceSynchronize();

            /*  Código en C++  */
}
```

Programación en CUDA - Ejecución de kernel

¿Cómo se ejecuta una rutina en cuda?

```
__global__ void KernelFunc(params){...}

main(){
                /*  Código en C++  */
    //reservar y transferir memoria a p
cant en x, y, z
    dim3   DimGrid(100, 50, 1);     // 5000 bloques
    dim3   DimBlock(4, 8, 8 );    // 256 threads por block
    size_t SharedMemBytes = 64; // 64 bytes de memoria shared
    KernelFunc<<< DimGrid, DimBlock, SharedMemBytes >>>(params);
    cudaDeviceSynchronize();

                /*  Código en C++  */
}
```

Programación en CUDA - Ejecución de kernel

¿Cómo se ejecuta una rutina en cuda?

```
__global__ void KernelFunc(params){...}

main(){
                /*  Código en C++  */
    //reservar y transferir memoria a p

    dim3   DimGrid(100, 50  );     // 5000 bloques
    dim3   DimBlock(4, 8, 8 );    // 256 threads por block
    size_t SharedMemBytes = 64; // 64 bytes de memoria shared
    KernelFunc<<< DimGrid, DimBlock, SharedMemBytes >>>(params);
    cudaDeviceSynchronize();
```

llamado al kernel

```
                /*  Código en C++  */
}
```

Programación en CUDA - Ejecución de kernel

¿Cómo se ejecuta una rutina en cuda?

```
__global__ void KernelFunc(params){...}

main(){
                /*  Código en C++  */
    //reservar y transferir memoria a p

    dim3   DimGrid(100, 50  );     // 5000 bloques
    dim3   DimBlock(4, 8, 8 );    // 256 threads por block
    size_t SharedMemBytes = 64; // 64 bytes de memoria shared
    KernelFunc<<< DimGrid, DimBlock, SharedMemBytes >>>(params);
    cudaDeviceSynchronize();
                                  sincronización
                                  explícita!!!
                /*  Código en C++  */
}
```

Programación en CUDA - Ejecución de kernel

¿Cómo se ejecuta una rutina en cuda?

defino el kernel

```
__global__ void KernelFunc(params){...}

main(){
            /*  Código en C++  */
    //reservar y transferir memoria a p
```

cant en x, y, z

```
    dim3   DimGrid(100, 50, 1);     // 5000 bloques
    dim3   DimBlock(4, 8, 8 );    // 256 threads por block
    size_t SharedMemBytes = 64; // 64 bytes de memoria shared
    KernelFunc<<< DimGrid, DimBlock, SharedMemBytes >>>(params);
    cudaDeviceSynchronize();
```

llamado al kernel

sincronización explícita!!!

```
            /*  Código en C++  */
}
```

Programación en CUDA

Espacios de declaración de variables :

- Host: Manejo de memoria de C
- Memoria global de la GPU:
  - Variables globales (_device_)
  - Punteros
- Memoria compartida:
  - Variables definidas en compilación (_shared_)
  - Allocamiento dinámico (extern _shared_)
- Memoria constante (_constant_)

```
__global__ void convolve (float *image)
__device__ float filter[N];
__shared__ float region[M];
extern __shared__ float region[];
__constant__ float filter[N];
```

Programación en CUDA

CUDA posee diversas keywords

- `threadIdx` (uint3) se accede con .x/.y/.z
- `blockIdx` (uint3) se accede con .x/.y/.z
- `blockDim` (dim3) se accede con .x/.y/.z
- `gridDim` (dim3) se accede con .x/.y/.z

```
region[threadIdx.x] = image[i];
```

Programación en CUDA

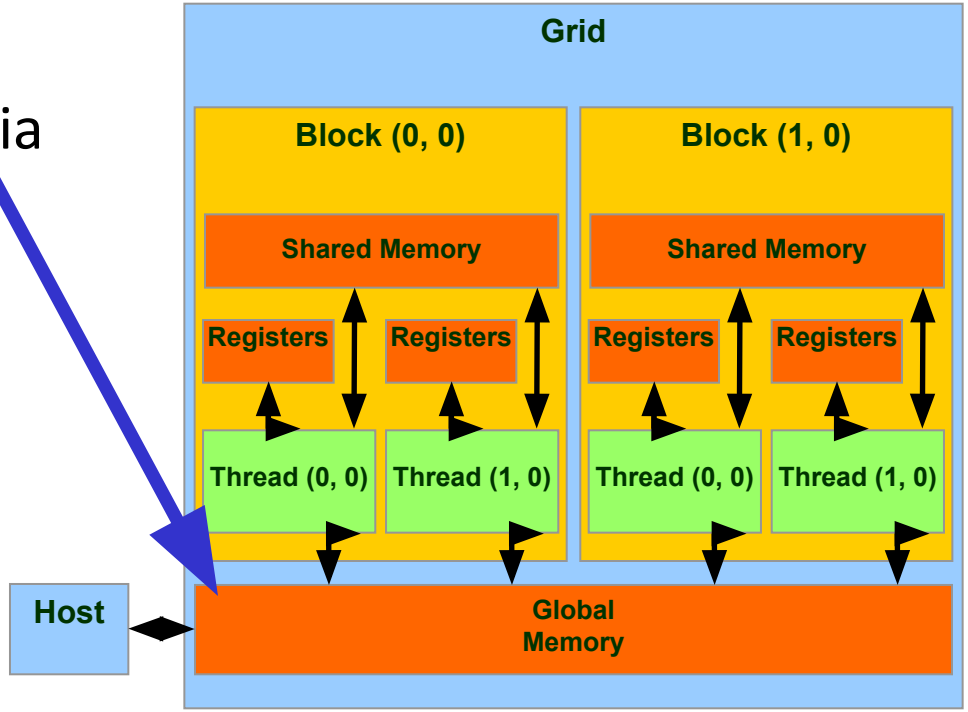
La API ofrece diversas funciones

- Para manejo de memoria
- Para transferencias
- Para manejo de las ejecuciones (lanzamiento)

```
// Allocate memoria en GPU
cudaMalloc((void**)& SDEVPTR, bytes)
```

CUDA Device Memory Allocation

- **`cudaMalloc()`**
  - Reserva espacio en la memoria global del device
  - Dos parámetros
    - Puntero
    - Tamaño de la memoria reservada
- **`cudaFree()`**
  - Libera la memoria asociada a un puntero en la mem. global del device



La reserva (y liberación de memoria global se realiza desde el host !!!

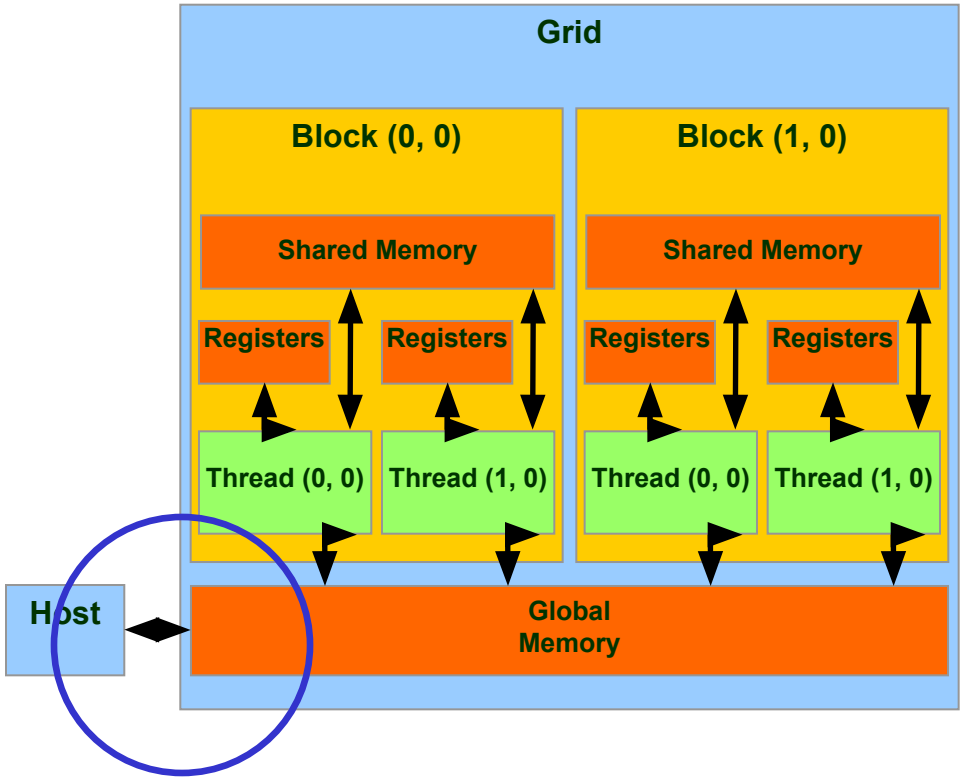
Programación en CUDA

- Código de ejemplo
  - Reservar una matriz de 64 * 64 floats (precisión simple).

```
TILE_WIDTH = 64;
float* SDEVPTR
int size = TILE_WIDTH *TILE_WIDTH* sizeof(float);

cudaMalloc((void**)& D_PTR, size);

...

cudaFree(D_PTR);
```

Transferencias de datos Host-Device

- `cudaMemcpy()`
  - Transferencia de datos
  - 4 parámetros
    - Puntero destino
    - Puntero origen
    - Número de bytes a copiar
    - Tipo de transferencia
      - Host to Host
      - Host to Device
      - Device to Host
      - Device to Device



- `Transferencias asincrónicas`

Programación en CUDA

- Código de ejemplo
  - Transferir una matriz de 64 x 64 floats (precisión simple)
    - **cudaMemcpyHostToDevice** y **cudaMemcpyDeviceToHost** son constantes simbólicas.

```
cudaMemcpy(D_PTR, HOST_DATA, size, cudaMemcpyHostToDevice);

cudaMemcpy(HOST_RES, D_PTR, size, cudaMemcpyDeviceToHost);
```

Lanzamiento

- Dimensiones de la grilla (3D): variable de tipo dim3
- Dimensiones de los bloques de threads (3D): variable de tipo dim3
- Opcionales:
  - Cantidad de memoria compartida por bloque
  - Identificador de stream

```
dim3 gridSize( 16, 16 );
dim3 blockSize( 32, 32 );
kernel<<<gridSize, blockSize, 0, 0>>>( ... );
kernel<<<32, 512>>>( ... );
```

Programación en CUDA

Funciones intrínsecas

- **cudaDeviceSynchronize()**: Sincroniza el host con el device (espera a que terminen todos los hilos). Se invoca en el host

- **__syncthreads()**: Sincroniza los threads del bloque. Se invoca dentro de un kernel.

- **clock_t clock()**: permite medir tiempos en el device.

Funciones matemáticas (más rápidas pero con posible diferencia en el valor):

- __sinf(x), __cosf(x), __log2f(x) …. __powf(x,y)

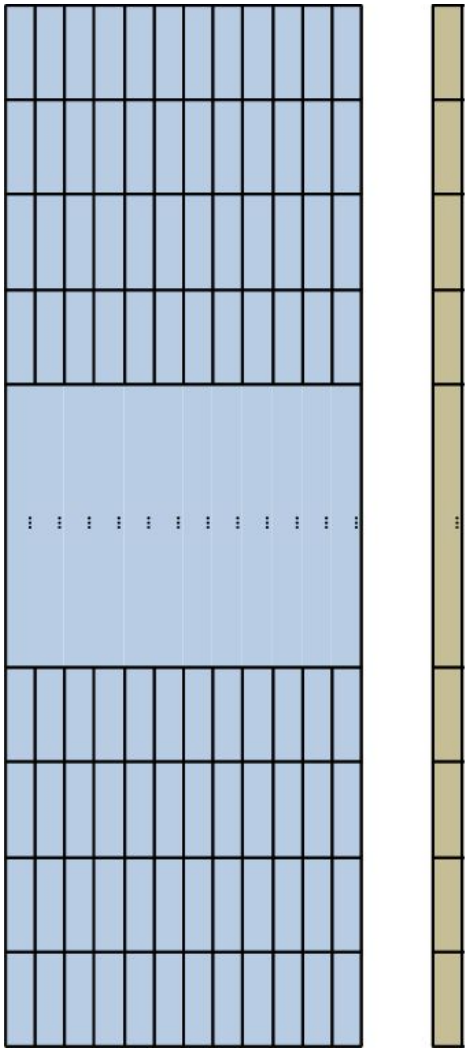
Funciones atómicas:

atomicAdd(), … atomicMin(), …

Programación en CUDA

Retomando los ejemplos de programación paralela básica :

- **Consideramos que se tiene una matriz con número grande de filas y se quiere obtener la suma por filas de la matriz (Almacenado por filas).**



Programación en CUDA

- **Ejemplo, suma por fila de los elementos de una matriz.**

```
void SumaFilaMatriz(int M, int N, float * Mh, float* Xh){

int size = M * N * sizeof(float), size2 = M*sizeof(float);
float* Md, *Xd;
…
// Allocate en device
cudaMalloc((void**)&Md, size);
cudaMalloc((void**)&Xd, size2);

// Inicializo matrices en el device
cudaMemcpy(Md, Mh, size, cudaMemcpyHostToDevice);
cudaMemset(Xd,0, size2);

Invocar el kernel que suma en GPU

// Traer resultado;
cudaMemcpy(Xh, Xd, size2, cudaMemcpyDeviceToHost);

// Free matrices en device
cudaFree(Md); cudaFree(Xd);
}
```

Kernel

```
// Suma por filas de una matriz
__global__ void SumaFilaMatrizKernel(int N, float* Md, float* Xd){
// Pvalue es usado para el valor intermedio
  float Pvalue = 0;
  int aux = threadIdx.x*N;
  for (int k = 0; k < N; ++k) {
     Pvalue = Pvalue + Md[aux+k];
  }
  Xd[threadIdx.x] = Pvalue;
}
```

Lanzamiento del kernel

```
// configuración de la ejecución
dim3 tamGrid(1, 1); //Grid dimensión
dim3 tamBlock(M,1,1); //Block dimensión
// lanzamiento del kernel
SumaFilaMatrizKernel<<<tamGrid, tamBlock>>>(N, Md, Xd);
```

Restricciones:

- Solo se pueden procesar matrices de tantas filas como threads en un bloque !!!!
- Un solo bloque, un solo multiprocesador computando.

Lanzamiento del kernel (2)

```
// configuración de la ejecución
int bloques = M / 128;
dim3 tamGrid(bloques, 1); //Grid dimensión
dim3 tamBlock(128, 1, 1); //Block dimensión
// lanzamiento del kernel
SumaFilaMatrizKernel<<<tamGrid, tamBlock>>>(N, Md, Xd);
```

Hay que hacer cambios en el Kernel !!!!

Kernel (2)

```
 // Suma por filas de una matriz
  __global__ void SumaFilaMatrizKernel(int N, float* Md, float* Xd){

// Pvalue es usado para el valor intermedio
  float Pvalue = 0;
  int aux = blockIdx.x*blockDim.x + threadIdx.x;
  for (int k = 0; k < N; ++k) {
     Pvalue = Pvalue + Md[aux*N+k];
  }
  Xd[aux] = Pvalue;
}
```

¿Funciona para todos los casos?

- Independientemente de la matriz da el resultado correcto pero ¿es la configuración óptima?
  - ¿Qué pasa si la matriz tiene “muchas” columnas y “pocas” filas ?
  - ¿Cómo podemos explotar mejor el paralelismo?

Lanzamiento del kernel (3)

```
// configuración de la ejecución
int chunk = 32; // Se asume N múltiplo de 32

dim3 tamGrid(1, M); //Grid dimensión
dim3 tamBlock(N/chunk,1,1); //Block dimensión
// lanzamiento del kernel
SumaFilaMatrizKernel<<<tamGrid, tamBlock>>>(N, Md, Xd);
```

Kernel (3)

```
// Suma por filas de una matriz
__global__ void SumaFilaMatrizKernel(int N, float* Md, float* Xd){
// Pvalue es usado para el valor intermedio
  float Pvalue = 0;

  int ini = blockIdx.x*N + threadIdx.x*(N/blockDim.x);
  int fin = ini + (N/blockDim.x);

  for (int k = ini; k < fin; k++) {
     Pvalue = Pvalue + Md[k];
  }

  Xd[blockIdx.y] = Xd[blockIdx.y] + Pvalue;
}
```

Esta implementación tiene varios problemas

- Varios hilos acceden con el mismo índice al vector Md (race condition)
  - **__atomicAdd(float* address, float val);**
- El acceso a memoria es bastante ineficiente (tener en cuenta en el práctico que viene!!!)
  - El patrón de acceso no es bueno para GPU

Programación en CUDA

Recordemos ….

```
__ atomicAdd(float* address, float val);
```

`atomicAdd()` funciona a partir de compute capabilities 2.0 !!!

Kernel (3)

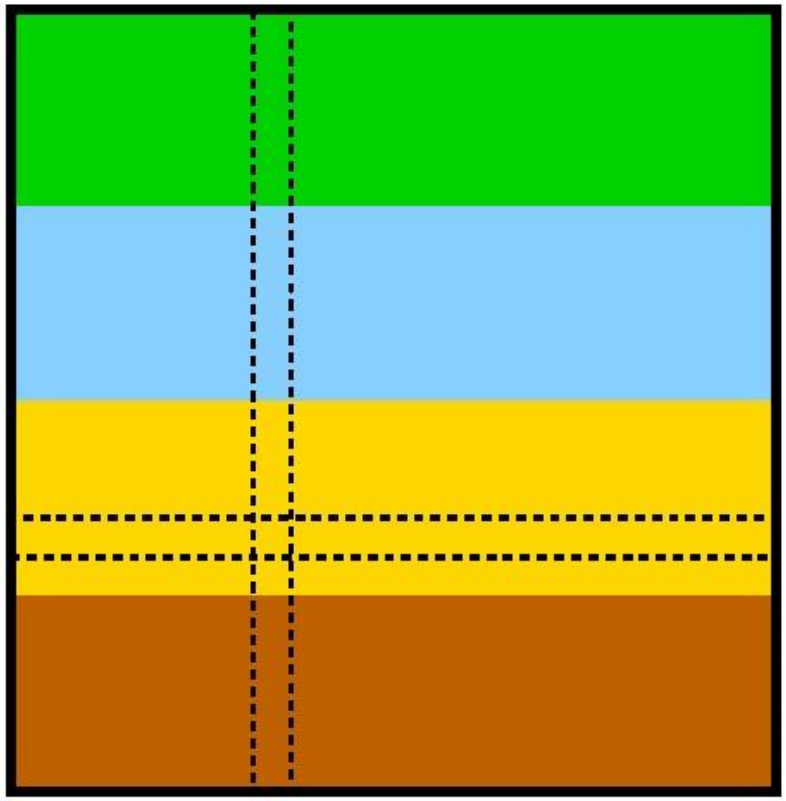
```
// Suma por filas de una matriz
__global__ void SumaFilaMatrizKernel(int N, float* Md, float* Xd){
// Pvalue es usado para el valor intermedio
  float Pvalue = 0;

  int ini = blockIdx.y*N + threadIdx.x*(N/blockDim.x);
  int fin = ini+ (N/blockDim.x);

  for (int k = ini; k < fin; k++) {
     Pvalue = Pvalue + Md[k];
  }

  atomicAdd(&(Xd[blockIdx.y]), Pvalue);
}
```

Kernel (3)

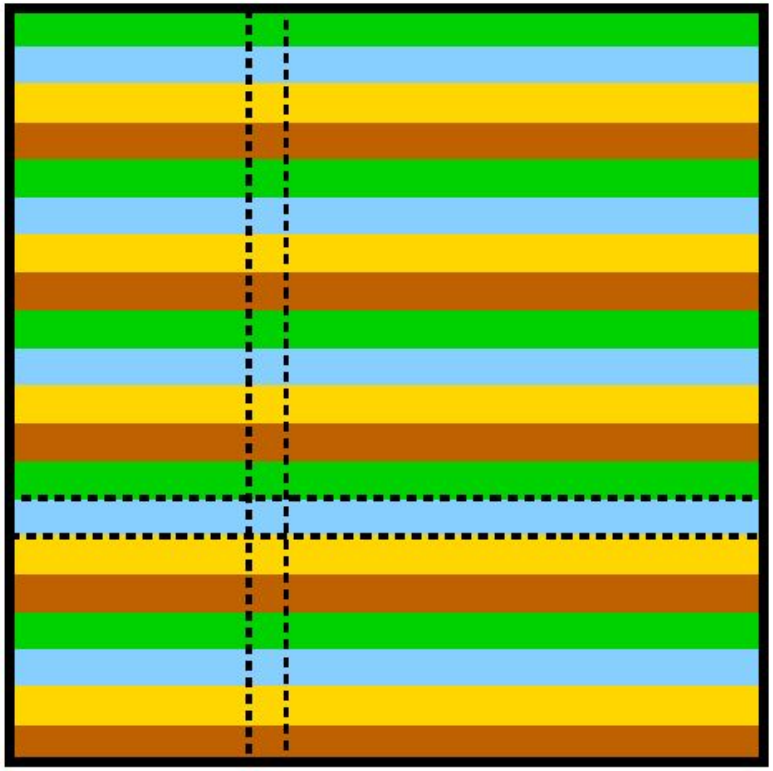
```
// Suma por filas de una matriz
__global__ void SumaFilaMatrizKernel(int N, float* Md, float* Xd){
// Pvalue es usado para el valor intermedio
  float Pvalue = 0;

  int ini = blockIdx.y*N + threadIdx.x*(N/blockDim.x);
  int fin = ini+ (N/blockDim.x);

  for (int k = ini; k < fin; k++) {
     Pvalue = Pvalue + Md[k];
  }

  atomicAdd(&(Xd[blockIdx.y]), Pvalue);
}
```

Conceptos Básicos de Programación Paralela

Distribución de datos/cálculos



Distribución por bloque



Distribución cíclica

Kernel (3.2)

```
// Suma por filas de una matriz
__global__ void SumaFilaMatrizKernel(int N, float* Md, float* Xd){
// Pvalue es usado para el valor intermedio
  float Pvalue = 0;

  int ini = blockIdx.y*N + threadIdx.x;
  int fin = (blockIdx.y+1)*N;
  int paso = blockDim.x;

  for (int k = ini; k < fin; k=k+paso) {
     Pvalue = Pvalue + Md[k];
  }

  atomicAdd(&(Xd[blockIdx.y]), Pvalue);
}
```

Lanzamiento del kernel (4)

Si quiero que un bloque procese más de una fila:

```
// configuración de la ejecución
filasBloque = 32
int chunk = 32; // Se asume M y N múltiplos de 32
dim3 tamGrid(1,M/chunk); //Grid dimensión
dim3 tamBlock(N/filasBloque,chunk); //Block dimensión
// lanzamiento del kernel
SumaFilaMatrizKernel<<<tamGrid, tamBlock>>>(M, N, Md, Xd);
```

Kernel (4)

```
// Suma por filas de una matriz
__global__void SumaFilaMatrizKernel (int M,int N,float*Md,float* Xd){
// Pvalue es usado para el valor intermedio
  float Pvalue = 0;

  int fila = blockIdx.x*blockDim.x+threadIdx.x;
  int col = blockIdx.y*blockDim.y +threadIdx.y

  for (int k = 0; k < pasos; ++k) {
     Pvalue = Pvalue + Md[fila*GridDim.x +col + k];
  }

  atomicAdd(&(Xd[fila]), Pvalue);

}
```